CAPSTONE PROJECT REPORT

Project Title: NLP Pipeline for Text Classification Using TF-IDF Vectorization and Logistic Regression

---

1. Project Overview

This project implements a Natural Language Processing (NLP) pipeline for basic text classification. The system preprocesses text, converts it into numerical features using **TF-IDF (Term Frequency-Inverse Document Frequency)** vectorization, and trains a supervised machine learning classifier to predict categories such as sentiment (positive/negative).

2. Objectives

The main objectives of the project are:

- **Preprocessing:** Clean raw text into a standardized format.
- **Vectorization:** Convert text into numeric vectors using TF-IDF.
- **Model Training:** Train a classification model using Logistic Regression.
- **Evaluation:** Measure model performance using standard metrics (Accuracy, Precision, Recall).
- **Persistence:** Save and reload the trained model for future predictions.

3. Dataset Description

The dataset consists of labelled text data suitable for supervised learning.

Column Name	Description
text	Raw input sentence or review
label	Target class/category (e.g., positive, negative)

Example records:

- *"I love this product!"* → positive
- *"Worst experience ever."* → negative

4. System Workflow (Pipeline)

The following steps outline the end-to-end process of the classification system:

- **Step 1: Data Loading:** Data is ingested from CSV files or manual entry.
- **Step 2: Text Cleaning:** Includes lowercase conversion, removing URLs, punctuation, digits, and extra whitespace.

- **Step 3: Train-Test Split:** Dataset is split (80% training / 20% testing) to ensure unbiased evaluation.
- **Step 4: Vectorization:** TF-IDF assigns weights based on word importance across the corpus.
- **Step 5: Model Training:** A Logistic Regression classifier is trained on the resulting sparse matrix.
- **Step 6: Evaluation:** Assessment via Confusion Matrix and F1-scores.
- **Step 7: Saving:** The pipeline is serialized as a .pkl file using **Joblib**.

5. Tools and Libraries Used

Library	Purpose
Pandas	Dataset loading and handling
NumPy	Numeric and matrix operations
Re	Regex-based text cleaning
Scikit-learn	Vectorization, model training, and evaluation
Joblib	Model saving and loading

6. Model Selection Justification

Why TF-IDF?

- Reduces the impact of frequent but unimportant words (like "the", "a").
- Captures the importance of rare words that carry more meaning.
- Extremely efficient for large text datasets.

Why Logistic Regression?

- **Efficiency:** Trains and predicts significantly faster than deep learning models.
- **Compatibility:** Performs exceptionally well with high-dimensional, sparse data produced by TF-IDF.
- **Baseline:** Provides a strong performance benchmark for text classification tasks.

7. Results (Example)

- **Accuracy:** ~0.80 (subject to dataset size).
- **Classification Report:** Precision and Recall are used to identify if the model is biased toward a specific class.

8. Conclusion & Future Improvements

This project successfully demonstrates a complete NLP pipeline. For future iterations, the model could be improved by:

- Implementing **Lemmatization** for better root-word analysis.
- Applying **Hyperparameter tuning** via GridSearchCV.
- Exploring Deep Learning architectures like **BERT** for higher accuracy.